ARM64 Memory Barriers — Technical Reference Workbook (CortexA72) 1
Based on ARM Documentation 1
Chapter 1 — ARMv8A Memory Ordering Model 1
1.1 Weakly Ordered Architecture 1
1.2 MultiCopy Atomicity 1
1.3 Visibility Domains 2
1.4 Ordering Rules 2
Chapter 2 — Memory Barriers: DMB, DSB, ISB 2
2.1 DMB — Data Memory Barrier 2
Purpose 3
Effect 3
Use Cases 3
2.2 DSB — Data Synchronization Barrier 3
Purpose 3
Effect 3
Use Cases 3
2.3 ISB — Instruction Synchronization Barrier 4
Purpose 4
Effect 4
Use Cases 4
2.4 Summary Table 4
Chapter 3 — Shareability Domains & Ordering Rules (CortexA72) 4
3.1 Shareability Types 5
3.2 Device Memory Ordering 5
3.3 Normal Memory Ordering 5
3.4 CortexA72 Specifics 5
Chapter 4 — Microarchitectural Behavior 6
4.1 FEQ (Fill/Evict Queue) 6
4.2 Snoop Logic 6

4.3 FrontEnd Pipeline 6
4.4 Why Barriers Are Mandatory 6
Chapter 5 — Barrier Cookbook (All UseCases) 7
5.1 Instruction Modification 7
5.2 CPU → DMA 7
5.3 DMA → CPU 7
5.4 CPU → CPU (Data) 7
5.5 CPU → CPU (Instructions) 8
5.6 MMIO Write 8
5.7 MMIO Read 8
5.8 DMA Descriptor Write 8
Appendix — Quick Reference Tables + Exercises 9
A.1 Quick Reference Table 9
A.2 Exercises 9

ARM64 Memory Barriers — Technical Reference Workbook (Cortex-A72)

Based on ARM Documentation

Chapter 1 — ARMv8-A Memory Ordering Model

1.1 Weakly Ordered Architecture

ARMv8-A is a **weakly ordered** architecture:

- Loads and stores may be reordered
- Speculation is allowed
- Cache hierarchy is not strictly ordered
- Ordering must be enforced explicitly using barriers

This applies to Cortex-A72 unless overridden by Device memory attributes.

1.2 Multi-Copy Atomicity

A write becomes visible to all observers **at the same instant** once it reaches **PoC**.

Cortex-A72 guarantees multi-copy atomicity **within a cluster**, and system-wide when memory is marked **Inner-shareable**.

1.3 Visibility Domains

ARM defines three visibility domains:

Domain	Meaning	Visible To

PoU	Point of Unification	All cores in cluster
PoC	Point of Coherence	DMA, other clusters
Device	MMIO ordering domain	Peripherals

Barriers control transitions between these domains.

1.4 Ordering Rules

ARM ordering rules depend on:

- Memory type (Normal vs Device)
- Shareability (Inner/Outer)
- Cacheability
- Speculation rules
- Barrier usage

Device memory is **strongly ordered**; Normal memory is **weakly ordered**.

Chapter 2 — Memory Barriers: DMB, DSB, ISB

This chapter synthesizes ARM documentation 102336_0101 and Cortex-A72 TRM behavior.

2.1 DMB — Data Memory Barrier

Purpose

Enforces **ordering** of memory accesses but does **not** guarantee completion.

Effect

- Prevents reordering of loads/stores
- Does not drain FEQ
- Does not complete cache maintenance
- Does not flush pipelines

Use Cases

- Ordering MMIO accesses
- Ordering Normal memory accesses
- SMP synchronization (ordering only)

2.2 DSB — Data Synchronization Barrier

Purpose

Enforces **completion** of all prior memory operations.

Effect

- Drains FEQ
- Completes DC CVAC/CIVAC/CVAU
- Completes snoops
- Ensures writebacks reach PoC
- Ensures instruction/data visibility transitions complete

Use Cases

- CPU→DMA
- DMA→CPU
- SMP visibility
- Cache maintenance completion
- MMIO ordering (DSB SY)

2.3 ISB — Instruction Synchronization Barrier

Purpose

Flushes the **front-end pipeline**.

Effect

- Flushes BTB

- Flushes RAS
- Flushes indirect predictor
- Flushes prefetch buffers
- Forces instruction refetch from PoU

Use Cases

- Instruction modification
- Changing system control registers
- Secondary core bring-up
- Context switching

2.4 Summary Table

Barrier	Ordering	Completion	Pipeline Flush	Use Case
DMB	Yes	No	No	Ordering only
DSB	Yes	Yes	No	Completion of memory ops
ISB	No	No	Yes	Instruction visibility

Chapter 3 — Shareability Domains & Ordering Rules (Cortex-A72)

3.1 Shareability Types

Type	Meaning
Non-shareable	No coherency between cores
Inner-shareable	Coherent within cluster
Outer-shareable	Coherent across clusters

Cortex-A72 uses **Inner-shareable** for SMP correctness.

3.2 Device Memory Ordering

Device-nGnRnE:

- No caching
- No speculation
- No merging
- No reordering
- DC/IC ignored
- Requires **DSB SY** for ordering

3.3 Normal Memory Ordering

Normal memory:

- Weakly ordered
- Requires barriers for correctness
- Cache maintenance required for visibility

3.4 Cortex-A72 Specifics

Cortex-A72:

- L1D/L2 are write-back
- Dirty lines may exist
- Snoop logic ensures coherency
- FEQ pipelines writebacks
- Barriers required for completion

Chapter 4 — Microarchitectural Behavior

4.1 FEQ (Fill/Evict Queue)

FEQ pipelines:

- Writebacks
- Snoop responses
- Evictions

DSB is required to drain FEQ.

4.2 Snoop Logic

Duplicate directory tracks dirty L1D lines.

Snoop completion is guaranteed only after **DSB ISH**.

4.3 Front-End Pipeline

ISB flushes:

- BTB
- RAS
- Indirect predictor
- Prefetch buffers
- Decode pipeline

4.4 Why Barriers Are Mandatory

Because:

- Cache maintenance is asynchronous
- Snoop responses are delayed
- Writebacks are pipelined
- Front-end speculation may fetch stale instructions

Chapter 5 — Barrier Cookbook (All Use-Cases)

5.1 Instruction Modification

```
DC   CVAU
IC   IVAU
DSB  ISH
ISB
```

5.2 CPU → DMA

```
DC   CVAC
DSB  ISH
```

5.3 DMA → CPU

```
DC   CIVAC
DSB  ISH
```

5.4 CPU → CPU (Data)

```
DC   CVAC
DSB  ISH
```

5.5 CPU → CPU (Instructions)

CPU0:

```
DC   CVAU
IC   IVAU
DSB  ISH
ISB
```

CPU1:

```
ISB
```

5.6 MMIO Write

```
STR
DSB SY
```

5.7 MMIO Read

```
DSB SY
LDR
```

5.8 DMA Descriptor Write

```
STR descriptor
DSB SY
STR start_register
DSB SY
```

Appendix — Quick Reference Tables + Exercises

A.1 Quick Reference Table

Scenario	Barrier Sequence
Instruction patching	CVAU → IVAU → DSB → ISB
CPU→DMA	CVAC → DSB
DMA→CPU	CIVAC → DSB
SMP data	CVAC → DSB
SMP instructions	CVAU → IVAU → DSB → ISB
MMIO write	STR → DSB SY
MMIO read	DSB SY → LDR

A.2 Exercises

1. Explain why DSB is required after DC CVAC.
2. Explain why ISB is required after IC IVAU.

3. Describe the difference between DMB and DSB.
4. Explain why Device-nGnRnE requires DSB SY.
5. Describe how FEQ affects barrier semantics.